

OWNER REFERENCE

How Star Player of the Week is built

A living document — updated as the app changes.

A plain reference for talking about this app with full confidence — what it's built with, how the data is organized, what's protected against, and how a change actually goes live.

01 WHAT IT'S BUILT WITH

FRONTEND

React + TypeScript

Built with Vite. Styled with Tailwind CSS on a small custom set of colors, spacing, and type sizes.

BACKEND

Firebase

No custom server, and no Cloud Functions. Firebase signs each browser in anonymously — nobody ever sees a sign-in screen — and stores the data.

DATABASE

Cloud Firestore

Data is stored as documents in named groups called "collections," not tables. The app gets live updates automatically, without asking for them.

HOSTING

Firebase Hosting

The site is hosted on Google's servers and deployed from the command line. There's no separate server to maintain.

UI PARTS

Radix UI + custom

Radix supplies accessible dialogs and toasts. Everything else — buttons, fields, badges — is built by hand to match the brand.

TESTING

Vitest + a Firestore test copy

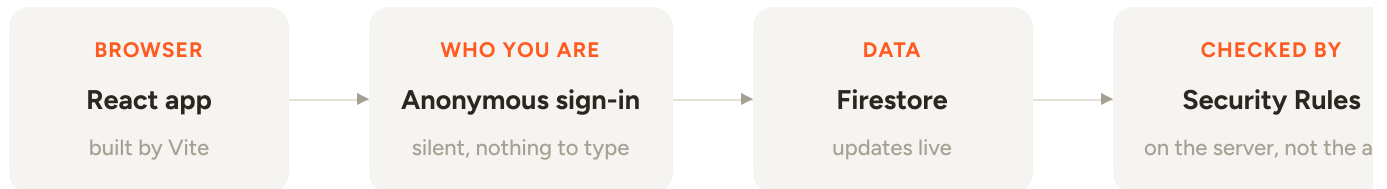
Unit tests check plain logic. A separate test suite runs the real security rules against a local test copy of the database, to prove access rules actually work.

CODE STORAGE

GitHub

One main branch. Every change is saved as a commit before it goes live.

02 HOW A REQUEST FLOWS THROUGH IT



Nobody signs in. The first time a browser opens the app, Firebase quietly hands it an anonymous ID — no email, no password, nothing to remember or lose. Tapping your name on the roster binds that ID to that name, and that binding is the whole of "who you are" here. It exists for one reason: without an identity the server itself can check, "this name is taken" would be a suggestion on screen that anyone could click straight past.

There's no backend server deciding permissions — the app talks to the database directly, and a separate file called **Security Rules** decides what a given browser is actually allowed to do. That matters: nobody can get more access just by changing what the app's own code does. Only changing the rules themselves changes what's allowed.

One name on the roster is different: whoever claims **KG** runs the session — opens and closes voting, pauses a week, calls the reveal — and is the only one who ever sees the totals. They don't vote themselves. There's no role to assign and no account to promote; claiming that name is the whole of it. The database learns who it is from a single record that KG's own browser writes, guarded so tightly that even the sitting host can't rewrite it to hand the seat to a browser that never actually claimed KG — the only two ways in are genuinely holding that claim yourself, or standing down and letting whoever holds it next take over. That second path is what lets a replaced phone recover: someone re-claims the name KG, then registers themselves as host in its place.

03 HOW THE DATA IS ORGANIZED

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
sotw_profiles	The roster — each teammate's name, plus their photo if they've added one. Nothing else	Everyone
sotw_claims	Which browser holds which name. Written once, when you tap your name, and never changed after	Everyone
sotw_meta/settings	Is voting open, which week it is, is the week paused, has it been revealed, who won	Everyone

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
sotw_meta/host	Which browser is running the session — written by whoever claims the name KG	Everyone
sotw_voters	That a person has voted this week. There is no field here for who they voted for	Everyone
sotw_tally	The running count of votes for each candidate — numbers only, no names attached to them	KG, once voting closes
sotw_stat_status	Each person's own "stats up" or "stats down" for the current week	Everyone
sotw_weekly_activity	That someone received at least one vote in a given week — what the streak badges are counted from	Everyone

Every name starts with `sotw_` — a simple prefix so this data can share a database with something else later without clashing.

There is no collection for "who I voted for", and that absence is the design. Your own pick is written to your browser's storage and nowhere else, so the link between a person and the name they chose never reaches the database at all. The cost is small and deliberate: clear your browser data and the app forgets which card to tick for you — the vote itself still stands, you just can't change it from a browser that no longer remembers it.

04 WHAT'S PROTECTED

✓ **One name, one person.** Tapping a name binds it to that browser, and the database allows that record to be created but never overwritten. So the first browser to take a name owns it, and nobody else can vote as that person — not by clicking past a greyed-out button, not by editing the app's own code. Freeing a name that's stuck on a phone somebody no longer has is a deliberate act, and only KG can do it.

✓ **Nobody can learn who voted for whom.** It is never recorded — not for teammates, not for KG, not for anyone reading the raw database. Two things get stored: that a person voted, and a running count for each candidate. KG is the only one who can read those counts, and only once voting has closed, so nobody can watch a number tick up and work out who just voted.

The second guarantee carries one accepted trade-off. Because the server holds no record of your pick, it can't check a vote being taken back against the vote you actually cast — it can only hold each change to a single point up or down. Someone hand-crafting requests could nudge a rival's count by one. Closing that would mean storing who voted for whom, which is precisely the thing this design refuses to do, so the one-point limit is where the line sits.

05 SHIPPING A CHANGE

Two separate steps, kept apart on purpose:

```
# app code — what people see and use
firebase deploy --only hosting

# security rules — who's allowed to do what
firebase deploy --only firestore:rules
```

Keeping these separate means a normal update to the app can never accidentally change who's allowed to do what. That only ever happens on purpose, as its own step.

06 WHAT'S HAPPENED SO FAR

Rebuilt from one big file

The app used to be a single large HTML file. It's now a proper React + TypeScript codebase, easier to maintain and grow. The old version is being phased out.

Security check-up

A full review found and fixed real gaps — including a way anyone could inflate vote counts.

Made the session controls self-serve

Adding, renaming and removing people, opening and closing voting, and recovering from a stuck state can now all be done from the app — no direct database edit needed.

Stats up or down, streaks, photos

Everyone says whether their stats are up before they vote, and only "up" goes on the poll. Photos, a badge for anyone who's received votes three weeks running, and a "pause this week" switch for holidays landed alongside it.

Stripped back to just the vote

Sign-in, accounts and the Blacfox Dollars wallet are gone, and every screen that wasn't the vote went with them. Open the app, tap your name, vote — that's the whole app now, and the rules got shorter and stricter for it.